

SIMATS ENGINEERING

CO5 ASSESSMENT TOOL 1

NAME : JAIVANT.S

REG NO : 192311326

COURSE NAME : DESIGN AND ANALYSIS OF ALGORITHMS

COURSE CODE : CSA0613

Q13. Clique Detection Problem — NP-Complete / Backtracking

Problem Statement

Given an undirected graph $G = (V, E)$ and an integer k , determine whether there exists a **clique of size k** .

A clique is a subset of vertices in which **every pair of vertices is directly connected by an edge**.

1. Constraints

- G contains n vertices.
- A valid clique must contain exactly k vertices.
- Every pair of selected vertices must be adjacent.
- For a clique of size k , the number of required edges is:

$$\frac{k(k-1)}{2}$$

- If $k > n$, a clique of size k is impossible.

The number of possible subsets of k vertices is:

$$\binom{n}{k}$$

Therefore, the search space can become very large as n increases.

2. Backtracking Approach

The algorithm generates subsets of vertices recursively.

At every step:

1. Select a vertex.
 2. Check whether it is adjacent to every vertex already selected.
 3. If it is adjacent, add it to the current clique.
 4. Continue until k vertices are selected.
 5. If a candidate is not adjacent to an existing vertex, **prune** that branch.
-

3. Algorithm

Clique($G, k, \text{current}, \text{start}$)

1. if $\text{size}(\text{current}) == k$:
 2. return TRUE

 3. for $v = \text{start}$ to n :
 4. if v is adjacent to every vertex in current :
 5. add v to current

 6. if Clique($G, k, \text{current}, v + 1$):
 7. return TRUE

 8. remove v from current

 9. return FALSE
-

4. Pruning Technique

The major pruning condition is:

if v is not adjacent to every vertex in current

 skip v

Such a vertex cannot form a clique with the already selected vertices, so its entire subtree is discarded.

Another useful pruning condition is:

$$|\text{current}| + (n - \text{start} + 1) < k$$

If the remaining vertices are insufficient to reach size k , the branch can immediately be stopped.

5. Example

Consider:

$$V = \{1, 2, 3, 4, 5\}$$

Suppose the graph contains edges:

$$(1, 2), (1, 3), (2, 3), (1, 4), (2, 4), (3, 4)$$

For $k = 3$, the algorithm may select:

$$\{1, 2, 3\}$$

Since:

- 1 is adjacent to 2,3
- 2 is adjacent to 1,3
- 3 is adjacent to 1,2

all pairs are connected.

Therefore:

Clique of size 3 exists

6. Complexity Analysis

In the worst case, the algorithm may examine many subsets of vertices.

The number of possible subsets of size k is:

$$\binom{n}{k}$$

Therefore, the worst-case running time is approximately:

$$O\left(\binom{n}{k} k^2\right)$$

if checking adjacency among selected vertices takes $O(k^2)$.

For unrestricted k , the problem has **exponential worst-case complexity**.

Space Complexity:

$$O(n)$$

for the recursion stack and current subset.

7. Why Clique Detection is NP-Complete

The **Clique Decision Problem** asks:

"Does graph G contain a clique of size at least k ?"

It is **NP-Complete** because:

1. **It belongs to NP:**

Given a set of k vertices, we can verify in polynomial time whether every pair is connected.

2. **It is NP-Hard:**

The Clique problem can be obtained through polynomial-time reductions from known NP-Complete problems such as **3-SAT**.

Hence:

Clique Decision Problem is NP-Complete

8. Effect of Constraints

The value of k strongly affects the search space.

- If k is small, fewer combinations need to be explored.
- If k is close to n , the number of possible combinations decreases.
- For intermediate values of k , $\binom{n}{k}$ can become extremely large.
- Strong pruning can eliminate many invalid candidates early.

Conclusion

The backtracking solution systematically generates possible vertex subsets while pruning candidates that violate the **clique adjacency constraint**. Although pruning improves practical performance, the worst-case search remains exponential. Since a proposed clique can be verified in polynomial time and the problem is NP-Hard, **Clique Detection is an NP-Complete problem**, making it an important example of combinatorial search and backtracking in DAA.
